THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 4

Rohan Garg(1024150187)

Q1) Create a class named *'Rectangle'* with two data members- *length* and *breadth* and a function to calculate the area which is 'length*breadth'. The class has three constructors which:
(a) having no parameter - values of both length and breadth are assigned zero.
(b) having two numbers as parameters - the two numbers are assigned as length and breadth respectively.
(c) having one number as parameter - both length and breadth are assigned that number.
Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

CODE:

```cpp
#include <iostream>
using namespace std;

class Rectangle {
    int length;
    int breadth;
public:
    // (1) Default constructor - no parameter
    Rectangle() {
        length = 0;
        breadth = 0;
    }
    // (2) Constructor with two parameters
    Rectangle(int l, int b) {
        length = l;
        breadth = b;
    }
    // (3) Constructor with one parameter
    Rectangle(int x) {
        length = x;
        breadth = x;
    }
```

```
    // Function to calculate area
    int area() {
        return length * breadth;
    }};
int main() {
    // Object with no parameter
    Rectangle r1;
    cout << "Area of r1: " << r1.area() << endl;
    // Object with one parameter
    Rectangle r2(5);
    cout << "Area of r2: " << r2.area() << endl;
    // Object with two parameters
    Rectangle r3(4, 6);
    cout << "Area of r3: " << r3.area() << endl;
    return 0;}
```

Output

```
Area of r1: 0
Area of r2: 25
Area of r3: 24


=== Code Execution Successful ===
```

Q2) Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

CODE:

```cpp
#include <iostream>
using namespace std;
class Rectangle {
    int length;
    int breadth;
public:
    // Constructor with default arguments
    // If no value is passed → both become 0
    // If one value is passed → breadth becomes 0
    Rectangle(int l = 0, int b = 0) {
        length = l;
        breadth = b;
        cout << "Constructor called (" << length << ", " << breadth << ")" << endl;
```

```cpp
    // Function to calculate area
    int area() {
        return length * breadth;}
    // Destructor
    ~Rectangle() {
        cout << "Destructor called for Rectangle ("
             << length << ", " << breadth << ")" << endl;}};
int main() {
    // Array of objects using different constructor forms
    Rectangle arr[3] = {
        Rectangle(),      // no parameter
        Rectangle(5),     // one parameter
        Rectangle(4, 6)   // two parameters
    };
    cout << "\nAreas of Rectangles:\n";
    for(int i = 0; i < 3; i++) {
        cout << "Area of object " << i+1 << " : "
             << arr[i].area() << endl;}
    return 0;}
```

Output

```
Constructor called (0, 0)
Constructor called (5, 0)
Constructor called (4, 6)

Areas of Rectangles:
Area of object 1 : 0
Area of object 2 : 0
Area of object 3 : 24
Destructor called for Rectangle (4, 6)
Destructor called for Rectangle (5, 0)
Destructor called for Rectangle (0, 0)


=== Code Execution Successful ===
```

Q3) Verify the following about *the destructor* by writing the program:

(i) The name should begin with a tilde sign (~) and must match the class name.

(ii) There cannot be more than one destructor in a class.

(iii) Destructors do not allow any parameter.

(iv) They do not have any return type, just like constructors.

CODE:

```
#include <iostream>
using namespace std;
class Demo {
public:
    // Constructor
    Demo() {
        cout << "Constructor called" << endl;}
    // Destructor
    ~Demo() {   // starts with ~ and matches class name
        cout << "Destructor called" << endl;
    }};
int main() {
    Demo obj1;   // Constructor runs here
    cout << "Inside main function" << endl;
    return 0;}    // Destructor runs automatically here}
```

Output

```
Constructor called
Inside main function
Destructor called


=== Code Execution Successful ===
```

Q4)When you do not specify any destructor in a class, compiler generates a default destructor and inserts it into your code.
Implement dynamic memory allocation. Use *new* and *delete* keywords.
(For integer variable, float variable, integer array, float array, class objects, Array of Objects)
CODE:

```
#include <iostream>
using namespace std;
class Demo {
public:
    int value;
    Demo(int v = 0) {
        value = v;
        cout << "Constructor called, value = " << value << endl;}
    ~Demo() {
        cout << "Destructor called, value = " << value << endl;
    }};
int main() {
    // Dynamic integer
    int *pInt = new int;
    *pInt = 10;
    cout << "Dynamic Integer: " << *pInt << endl;
    delete pInt;   // free memory
    cout << endl;
    // Dynamic float
    float *pFloat = new float;
    *pFloat = 5.5;
    cout << "Dynamic Float: " << *pFloat << endl;
    delete pFloat;
    cout << endl;
```

```
    // Dynamic integer array
    int *arrInt = new int[3];
    arrInt[0] = 1;
    arrInt[1] = 2;
    arrInt[2] = 3;

    cout << "Dynamic Integer Array: ";
    for(int i = 0; i < 3; i++) {
        cout << arrInt[i] << " ";}
    cout << endl;
    delete[] arrInt;   // use delete[] for arrays
    cout << endl;
    // Dynamic float array
    float *arrFloat = new float[2];
    arrFloat[0] = 1.1;
    arrFloat[1] = 2.2;
    cout << "Dynamic Float Array: ";
    for(int i = 0; i < 2; i++) {
        cout << arrFloat[i] << " ";}
    cout << endl;
    delete[] arrFloat;
    cout << endl;
```

```
    cout << endl;
    // Dynamic class object
    Demo *obj = new Demo(100);
    delete obj;   // destructor called
    cout << endl;
    //Dynamic array of objects
    Demo *objArr = new Demo[2] { Demo(10), Demo(20) };
    delete[] objArr;   // destructors called in reverse order
    return 0;}
```

Output

```
Dynamic Integer: 10

Dynamic Float: 5.5

Dynamic Integer Array: 1 2 3

Dynamic Float Array: 1.1 2.2

Constructor called, value = 100
Destructor called, value = 100

Constructor called, value = 10
Constructor called, value = 20
Destructor called, value = 20
Destructor called, value = 10


=== Code Execution Successful ===
```